

Inventory Management System for B2B SaaS

Submitted by: Aakanksha Sah

Part 1: Code Review & Debugging

A previous intern wrote this API endpoint for adding new products. Something is wrong - the code compiles but doesn't work as expected in production.

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

Task Performed:

1. Issues identified:

- a. There is no checks/ input monitoring for missing fields, invalid types or negative Values.
- b. Code adds products even when SKU exists. Same product being added twice can lead to problems.
- c. There is missing initial_quantity where code uses datainitialquantity but request.json lacks it.

- d. There is no error handling part i.e no try-catch for if inventory add fails after product commit. This can lead to data inconsistency.
- e. No Authorization for who can add products. Anybody can based on the code.
- f. Since products exists in multiple warehouses, when WarehouseId is passed it adds to one only, but doesn't prevent duplicates elsewhere.

2. Impact in Production

- a. Invalid data can crash the app or corrupt the database.
- b. Unauthorized access can lead to many serious data risks including data tampering.
- c. There could be inconsistent multi-warehouse tracking for products.
- d. KeyError in missing quantity can crash endpoints.

Fixed Code:

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    #Creating Required fields
    required=['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    for field in required:
        if field not in data or data[field] is None:
            return {"error": f"Missing {field}"}, 400

    # Checking if SKU exists
    if Product.query.filter_by(sku=data['sku']).first():
        return {"error": "SKU already exists"}, 409

    try:
        #Saving both together
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=float(data['price']))
        db.session.add(product)
        db.session.flush()

        inventory=Inventory(
            Product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=data['initial_quantity']
        )
```

```

db.session.add( inventory)
db.session.commit()
return {"message": "Product created", "product_id": product.id}
except Exception as e:

db.session.rollback()

return {"error" : str(e)}, 500

```

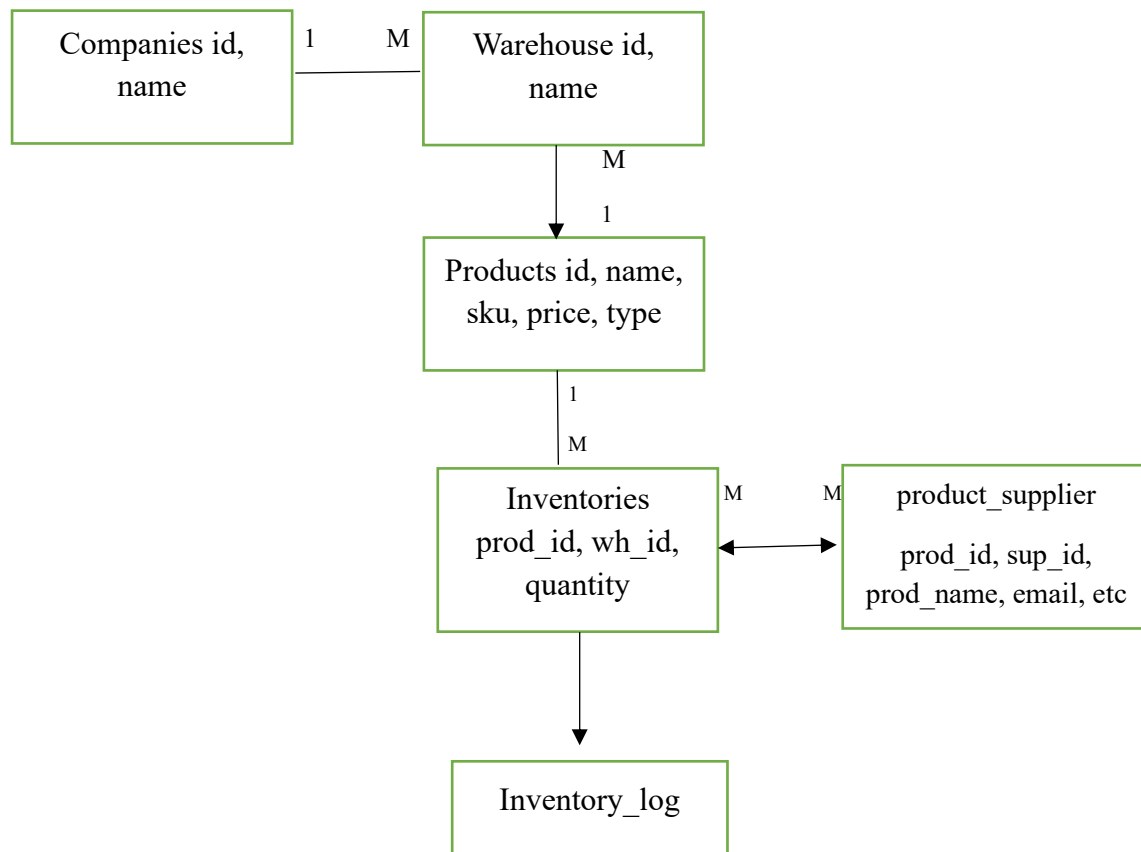
Part 2: Database Design (25 minutes)

Based on the requirements below, design a database schema. **Note:** These requirements are intentionally incomplete - you should identify what's missing.

Given Requirements:

- Companies can have multiple warehouses
- Products can be stored in multiple warehouses with different quantities
- Track when inventory levels change
- Suppliers provide products to companies
- Some products might be "bundles" containing other products

Task Performed:



SQL Code:

```
CREATE TABLE companies(
```

```
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(255) NOT NULL
```

```
);
```

```
CREATE TABLE warehouse{
```

```
    id SERIAL PRIMARY KEY,
```

```
    company_id INT REFERENCES COMPANIES(id),
```

```
    name VARCHAR(255) NOT NULL
```

```
    UNIQUE(company_id, name)
```

```
);
```

```
CREATE TABLE products(
```

```
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(255) NOT NULL,
```

```
    sku VARCHAR(100) NOT NULL UNIQUE,
```

```
    price DECIMAL(10,2) NOT NULL,
```

```
    type VARCHAR(50) DEAFULT 'standard',
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
CREATE TABLE suppliers{
```

```
    id SERIAL PRIMARY LEY,
```

```
    name VARCHAR(255) NOT NULL,
```

```
    email VARCHAR(255),
```

```
);
```

```
CREATE TABLE product_suppliers{
```

```
    product_id INT REFERENCES products(id) ON DELETE CASCADE,
```

```
    warehouse_id INT REFERENCES suppliers(id) ON DELETE CASCADE,
```

```
    PRIMARY KEY(product_id, supplier_id)
```

```
);
```

```
CREATE TABLE inventories(
```

```
    id SERIAL PRIMARY KEY,
```

```

        product_id INTEGER REFERENCES products(id) ON DELETE CASCADE,
        warehouse_id INT REFERENCES suppliers(id),
        quantity INT DEFAULT 0 CHECK (quantity>=0),
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
        UNIQUE(product_id, warehouse_id)
);
CREATE TABLE inventory_logs(
        id SERIAL INT PRIMARY KEY,
        inventory_id INT REFERENCES inventories(id) ON DELETE CASCADE,
        change_type VARCHAR(20) NOT NULL,
        quantity_change INT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE bundle_items (
        bundle_id INT REFERENCES products(id) ON DELETE CASCADE,
        component_id INT REFERENCES product(id) ON DELETE CASCADE,
        Quantity INT NOT NULL CHECK (quantity>=0),
        PRIMARY KEY (bundle_id, component_id)
);

```

Design:

- Kept sku as unique global id
- Created table inventory_logs to track all changes.
- Used inventories UNIQUE(prod_id, wh_id) so as to keep one quantity per product or in warehouse.
- Used Decimal upto 2 places (DECIMAL(10,2)) to keep up the money precision.
- Used type field to use different stock thresholds.

Gaps Identification

- What defined the product type value and their lowest stock value?
- Can different companies share the suppliers or products or not?
- Maximum quantity limit of each warehouses.
- At what value of “recent sales activity” should there be alerts?
- Should there be monitoring of who makes the inventory changes?

Part 3: API Implementation (35 minutes)

Implement an endpoint that returns low-stock alerts for a company.

Business Rules (discovered through previous questions):

- Low stock threshold varies by product type
- Only alert for products with recent sales activity
- Must handle multiple warehouses per company
- Include supplier information for reordering

Implementation code using python/flask

```
@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
def low_stock_alerts(company_id):
    THRESHOLDS = {'standard': 20, 'perishable': 10} # Rule 1: Threshold by type

    try:
        # EDGE CASE 1
        warehouses = db.session.query(Warehouse).filter_by(company_id=company_id).all()
        if not warehouses:
            return jsonify({'alerts': [], 'total_alerts': 0}), 200

        alerts = []

        for warehouse in warehouses:

            low_items = db.session.query(Inventory).outerjoin(Product).filter(
                Inventory.warehouse_id == warehouse.id,
                Inventory.quantity < THRESHOLDS.get(Product.type, 20) # Default 20
            ).all()

            for inv in low_items:
                product = inv.product

                # EDGE CASE 2
                recent_sales = db.session.query(func.sum(Sale.quantity)).filter(
                    Sale.product_id == product.id,
                    Sale.created_at >= datetime.now() - timedelta(days=30)
                ).scalar() or 0

                if recent_sales == 0:
                    continue # Rule 2: Only recent activity

                # EDGE CASE 3
                avg_daily = recent_sales / 30.0
                days_out = 999 if avg_daily == 0 else int(inv.quantity / avg_daily)

                # EDGE CASE 4
```

```

supplier = db.session.query(Supplier).join(ProductSupplier).filter(
    ProductSupplier.product_id == product.id
).first()

alert = {
    'product_id': product.id,
    'product_name': product.name,
    'sku': product.sku,
    'warehouse_id': warehouse.id,
    'warehouse_name': warehouse.name,
    'current_stock': inv.quantity,
    'threshold': THRESHOLDS.get(product.type, 20),
    'days_until_stockout': days_out,
    'supplier': {
        'id': supplier.id if supplier else None,
        'name': supplier.name if supplier else 'No supplier assigned',
        'contact_email': supplier.email if supplier else None
    }
}
alerts.append(alert)

# Sort by urgency (bonus!)
alerts.sort(key=lambda x: x['days_until_stockout'])

return jsonify({
    'alerts': alerts,
    'total_alerts': len(alerts)
}), 200

except Exception as e:
    # EDGE CASE 5:
    return jsonify({'error': 'Service unavailable', 'details': str(e)}), 500

```

Assumptions made are:

- Sales table exists:sales(Product_id, quantity, date)
- Product types are standard-20, perishable=10
- Recent-30 days

Edge cases considered:

- If company has no warehouses(1), then it will give empty responses. This also ensures no crashing.
- If Any product has no recent sales(2) , then no need to bring those products in use, just “continue” will happen.
- Calculating average daily sales of any product(3)
- What if no supplier has been assigned to a product(4), this could mean missing data, to handle that it’ll be given “No supplier assigned”.
- Most important, if database error occurs.(5)